Capítulo 1 - Conceitos básicos em análise algorítmica

Projeto e Análise de Algoritmos

Professor: Henrique Viana Oliveira, henriq.viana@uece.br

Exercício: 1.1

Seja $A[1..60] = 11, 12, ..., 70$. Quantas comparações são realizadas pelo Algoritmo BINARY-SEARCH quando busca pelos seguintes valores de x?

(a) 33.

(b) 7.

(c) 70.

(d) 77.

Exercício: 1.2

Seja $A[1..2000] = 1, 2, ..., 2000$. Quantas comparações são realizadas pelo Algoritmo BINARYSEARCH quando busca pelos seguintes valores de x?

(a) -3.

(b) 1.

(c) 1000.

(d) 4000.

Exercício: 1.3

Desenhe a árvore de decisão para o algoritmo de busca binária com uma entrada de

(a) 12 elementos.

(b) 17 elementos.

(c) 25 elementos.

(d) 35 elementos.

Exercício: 1.4

Mostre que a altura da árvore de decisão para busca binária é $\lfloor logn \rfloor$.

Exercício: 1.5

Ilustre a operação do Algoritmo SELECTIONSORT no vetor $A = (45, 33, 24, 45, 12, 12, 24, 12)$. Quantas comparações são realizadas pelo algoritmo?

Exercício: 1.6

Considere modificar o Algoritmo SELECTIONSORT como mostrado no Algoritmo MODSELECTIONSORT.

(a) Qual é o número mínimo de atribuições de elementos realizadas pelo Algoritmo MODSELECTIONSORT? Quando esse número mínimo é obtido?

(b) Qual é o número máximo de atribuições de elementos realizadas Algoritmo MODSELECTIONSORT? Note que cada troca é implementada usando três atribuições de elementos. Quando esse número máximo é obtido?

Algorithm 1.16 MODSELECTIONSORT
Input: An array $A[1..n]$ of n elements.
Output: $A[1..n]$ sorted in nondecreasing order.

```
for i ← 1 to n − 1
    for j ← i + 1 to n
        if A[j] < A[i] then interchange A[i] and A[j]
    end for
end for
```

Exercício: 1.7

Ilustre a operação do Algoritmo INSERTIONSORT no vetor $A = (30, 12, 13, 13, 44, 12, 25, 13)$. Quantas comparações são realizadas pelo algoritmo?

Exercício: 1.8

Quantas comparações são realizadas pelo Algoritmo INSERTIONSORT quando apresentado com a entrada $A = (4, 3, 12, 5, 6, 7, 2, 9)$?

Exercício: 1.10

Qual algoritmo é mais eficiente: Algoritmo INSERTIONSORT ou Algoritmo SELECTIONSORT? E se o vetor de entrada consiste de registros muito grandes? Explique.

Exercício: 1.11

Ilustre a operação do Algoritmo BOTTOMUPSORT no vetor $A[1..16] = (11, 12, 1, 5, 15, 3, 4, 10, 7, 2, 16, 9, 8, 14, 13, 6)$. Quantas comparações são realizadas pelo algoritmo?

Exercício: 1.12

Ilustre a operação do Algoritmo BOTTOMUPSORT no vetor $A[1..11] = (2, 17, 19, 5, 13, 11, 4, 8, 15, 12, 7)$. Quantas comparações são realizadas pelo algoritmo?

Exercício: 1.13

Dê um vetor $A[1..8]$ de inteiros em que o Algoritmo BOTTOMUPSORT realiza

(a) O número mínimo de comparações de elementos.

(b) O número máximo de comparações de elementos.

Exercício: 1.15

Expresse as seguintes funções em termos da notação Θ.

(a) $2n + 3log^{100}n$.

(b) $7n^3 + 1000nlogn + 3n$.

(c) $3n^{1.5} + (\sqrt{n})^3 logn$.

(d) $2^n + 100^n + n!$.

Exercício: 1.16

Expresse as seguintes funções em termos da notação Θ.

(a) $18n^3 + logn^8$.

(b) $(n^3 + n)/(n + 5)$.

(c) $log^2 n + \sqrt{n} + loglogn$.

(d) $n!/2^n + n^n$.

Exercício: 1.17

Considere o algoritmo de ordenação mostrado abaixo, que é chamado BUBBLESORT.

(a) Qual é o número mínimo de comparações de elementos realizadas pelo algoritmo? Quando esse mínimo é obtido?

(b) Qual é o número máximo de comparações de elementos realizadas pelo algoritmo? Quando esse máximo é obtido?

(c) Qual é o número mínimo de atribuições de elementos realizadas pelo algoritmo? Quando esse mínimo é obtido?

(d) Qual é o número máximo de atribuições de elementos realizadas pelo algoritmo? Quando esse máximo é obtido?

(e) Expresse o tempo de execução do Algoritmo BUBBLESORT em termos das notações O e Ω.

(f) O tempo de execução do algoritmo pode ser expresso em termos da notação Θ? Explique.

Algorithm 1.17 BUBBLESORT

Input: An array $A[1..n]$ of n elements.

Output: $A[1..n]$ sorted in nondecreasing order.

```
i ← 1;    sorted ← false
while i ≤ n − 1 and not sorted
    sorted ← true
    for j ← n downto i + 1
        if A[j] < A[j − 1] then
            interchange A[j] and A[j − 1]
            sorted ← false
        end if
    end for
    i ← i + 1
end while
```

Exercício: 1.20

Prove que $\sum_{j=1}^{n} j^k$ é $O(n^{k+1})$ e $\Omega(n^{k+1})$, onde k é um inteiro positivo. Conclua que ela é $\Theta(n^{k+1})$.

Exercício: 1.22

Mostre que $n^{100} = O(2^n)$, mas $2n \neq O(n^{100})$.

Exercício: 1.23

Mostre 2^n não é $\Theta(3^n)$.

Exercício: 1.24

É verdade que $n! = \Theta(n^n)$? Prove sua resposta.

Exercício: 1.25

É verdade que $2^{n^2} = \Theta(2^{n^3})$? Prove sua resposta.

Exercício: 1.27

A função $\lfloor logn \rfloor!$ é $O(n), \Omega(n)$ e $\Theta(n)$? Prove sua resposta.

Exercício: 1.32

Considere o Algoritmo COUNT6 cuja entrada é um inteiro positivo n.

(a) Quantas vezes a linha 6 é executada?

(b) Qual das notações O e Θ é mais apropriada para expressar a complexidade de tempo do algoritmo? Explique.

(c) Qual a complexidade de tempo do algoritmo?

Algorithm 1.18 COUNT6

```
1. comment: Exercise 1.32
2. count ← 0
3. for i ← 1 to ⌊log n⌋
4.     for j ← i to i + 5
5.         for k ← 1 to i^2
6.             count ← count + 1
7.         end for
8.     end for
9. end for
```

Exercício: 1.33

Considere o Algoritmo COUNT7 cuja entrada é um inteiro positivo n.

(a) Qual é o número máximo de vezes que a linha 6 é executada quando n é uma potência de 2?

(b) Qual a complexidade de tempo do algoritmo expressa na notação O?

(c) Qual a complexidade de tempo do algoritmo expressa na notação Ω?

(d) Qual das notações O e Θ é mais apropriada para expressar a complexidade de tempo do algoritmo? Explique brevemente.

Algorithm 1.19 COUNT7

```
comment: Exercise 1.33
count ← 0
for i ← 1 to n
    j ← ⌊n/2⌋
    while j ≥ 1
        count ← count + 1
        if j is odd then j ← 0  else j ← j/2
    end while
end for
```

Exercício: 1.34

Considere Algoritmo COUNT8 cuja entrada é um inteiro positivo n.

(a) Qual é o número máximo de vezes que a linha 7 é executada quando n é uma potência de 2?

(b) Qual é o número máximo de vezes que a linha 7 é executada quando n é uma potência de 3?

(c) Qual a complexidade de tempo do algoritmo expressa na notação O?

(d) Qual a complexidade de tempo do algoritmo expressa na notação Ω?

(e) Qual das notações O e Θ é mais apropriada para expressar a complexidade de tempo do algoritmo? Explique brevemente.

Algorithm 1.20 COUNT8

```
1.  comment: Exercise 1.34
2.  count ← 0
3.  for i ← 1 to n
4.      j ← ⌊n/3⌋
5.      while j ≥ 1
6.          for k ← 1 to i
7.              count ← count + 1
8.          end for
9.          if j is even then j ← 0 else j ← ⌊j/3⌋
10.     end while
11. end for
```

Exercício: 1.35

Escreva um algoritmo para encontrar o máximo e o mínimo de uma sequência de n inteiros armazenados no vetor $A[1..n]$ tal que sua complexidade de tempo seja

(a) $O(n)$.

(b) $\Omega(nlogn)$.

Exercício: 1.36

Seja $A[1..n]$ um vetor de inteiros distintos, onde $n > 2$. Dê um algoritmo de tempo $O(1)$ que descobre se um elemento de A é nem o máximo nem o mínimo.

Exercício: 1.38

Dê um algoritmo que avalia uma entrada polinomial $a_n x^n + a_{n-1}x^{n-1} + \cdots + a_1 x + a_0$, para um dado valor de x em tempo

(a) $\Omega(n^2)$.

(b) $O(n)$.

Exercício: 1.41

Sejam m e n dois inteiros positivos. O maior divisor comum (*greatest commom divisor*) de m e n, denotado por $gcd(m, n)$, é o maior inteiro que divide ambos m e n. Por exemplo $gcd(12, 18) = 6$. Considere o Algoritmo EUCLID mostrado abaixo, para computar $gcd(m, n)$.

(a) Faz alguma diferença se na primeira chamada de $gcd(m, n)$ acontece que $n < m$? Explique.

(b) Prove a corretude do Algoritmo EUCLID. (Dica: Use o seguinte teorema: Se r divide ambos m e n, então r divide $m - n$).

(c) Mostre que o tempo de execução do Algoritmo EUCLID é máximo se m e n são dois números consecutivos na sequência de Fibonacci definida por $f_1 = f_2 = 1; f_n = f_{n-1} + f_{n-2}$ para $n > 2$.

(d) Analise o tempo de execução do Algoritmo EUCLID em termos de n, assumindo que $n \geq m$.

(e) A complexidade de tempo do Algoritmo EUCLID pode ser expresso usando a notação Θ? Explique.

Exercício: 1.42

Encontre a complexidade de tempo do Algoritmo EUCLID discutido no Exercício 1.41 medida em termos do tamanho da entrada. A complexidade é logarítmica, linear, exponencial? Explique.

Algorithm 1.21 EUCLID

Input: Two positive integers m and n.

Output: $gcd(m, n)$.

```
comment: Exercise 1.41
repeat
    r ← n mod m
    n ← m
    m ← r
until r = 0
return n
```